UD2 Examen

Estructuras de Control y Depuración

Ejercicio 1: El Escáner de Seguridad (Dificultad: Baja)

package examen2;

import java.util.Scanner;

/*

Alejandro Pozo Portillo

Ejercicio 1: El Escáner de Seguridad (Dificultad: Baja)

(package examen2;

import java.util.Scanner;

/*

Alejandro Pozo Portillo

Ejercicio 1: El Escáner de Seguridad (Dificultad: Baja)

(Vinculado a CE a, CE e, CE g)

En una estación espacial, el terminal de control clasifica a las naves según su código de autorización numérico (un número entero del 1 al 4):

- 1: Nave Aliada Imprime: "Acceso concedido: Sector Central".
- 2: Nave Comercial Imprime: "Acceso concedido: Hangar de Carga".

• 3: Nave Desconocida Imprime: "Acceso restringido: Enviar escolta".

• 4: Nave Hostil Imprime: "¡ALERTA! Iniciar protocolos de defensa".

• Cualquier otro número Imprime: "Código no válido".

Se pide:

1. Escribe un programa en Java que solicite al usuario el código numérico y procese la respuesta utilizando la estructura switch.

2. Si la nave es Comercial (código 2), añade una condición adicional (if / else) que pida el tonelaje de carga (número entero). Si supera los 500 toneladas, debe mostrar el mensaje adicional: "Requiere inspección aduanera".

3. Comenta brevemente la función de cada bloque principal.

*/

```
public class Examen2 {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        //Le pedimos que seleccione una opcion
        System.out.println("Elige una opcion");
        System.out.println("1: Nave Aliada");
        System.out.println("2: Nave Comercial");
        System.out.println("3: Nave Desconocida");
        System.out.println("4: Nave Hostil");
        int opcion = sc.nextInt();
```

```
        //Creamos el swtich para que la seleccion escoja una opcion
        switch (opcion) {


            //Pinta en caso de que ponga 1
            case 1 ->
                System.out.println("Acceso concedido: Sector Central");
            //Pinta en caso de que ponga 2
            case 2 -> {
                //Pedimos el peso entero en toneladas
                System.out.println("¿Cuantas toneladas pesa la carga? (En numero entero)");
                int peso = sc.nextInt();
                //Si el peso es menor a 500 pone el mensaje predeterminado
                if (peso < 500) {
                    System.out.println("Acceso concedido: Hangar de Carga");
                } //Si el peso supera a 500 el mensaje predeterminado mas el extra
                else {
                    System.out.println("Acceso concedido: Hangar de Carga, pero requiere inspeccion
aduanera");
                }
            }
            //Pinta en caso de que ponga 3
            case 3 ->
                System.out.println("Acceso restringido: Enviar escolta");
            //Pinta en caso de que ponga 4
            case 4 ->
```

```
            System.out.println("¡ALERTA! Iniciar protocolos de defensa");
        //Pinta en caso de que ponga una opcion incorrecta
        default ->
            System.out.println("Codigo no valido");

    }
  }
}
```

Ejercicio 2: El Generador de Frecuencias de Radio (Dificultad: Media)

```
package examen2;

/*
Alejandro Pozo Portillo

Ejercicio 2: El Generador de Frecuencias de Radio (Dificultad: Media)

(Vinculado a CE b, CE c, CE e, CE g)

Para sintonizar una emisora de emergencia, el sistema debe probar una serie de frecuencias de radio numéricas desde la frecuencia 100 hasta la 150 (de 1 en 1).

Se pide: Escribe un programa utilizando un bucle que analice las frecuencias en dicho rango bajo las siguientes reglas de negocio:

Las frecuencias divisibles entre 5 se consideran "Frecuencias saturadas" y deben saltarse sin imprimirse (utiliza la sentencia adecuada para saltar a la siguiente iteración).

Si la frecuencia es exactamente 137, se ha encontrado la "Frecuencia de Socorro". El programa debe mostrar por pantalla "¡Frecuencia de socorro 137 hallada! Deteniendo el escaneo." y interrumpir inmediatamente el bucle.

Para todas las demás frecuencias válidas, se debe imprimir: "Sintonizando frecuencia: [número]".
```

```
 */
public class Examen2_1 {


    public static void main(String[] args) {
        //Añadimos las variables, finFre es el final de la frecuencia
        int finFre = 150;
        //y inicioFre el inicio de la frecuencia
        int inicioFre = 100;
        //Si inicioFre es mayor que finFre acaba el bucle
        while (inicioFre < finFre) {
            //Para que añada 1 a inicioFre
            ++inicioFre;
            //Si es divisible entre 5 con resto 0 se salta
            if (inicioFre % 5 == 0) {
                continue;
            }
            //Si es el numero 137 se para el bucle mostrando tambien frecuencia de socorro
            if (inicioFre == 137) {
                System.out.println("Frecuencia de Socorro");
                break;
            }
            System.out.println(inicioFre);
        }
    }
}
```

Ejercicio 3: Control de Recarga e Inspección de Código (Dificultad: Alta)

```
package examen2;

import java.util.Scanner;

/*
Alejandro Pozo Portillo

Ejercicio 3: Control de Recarga e Inspección de Código (Dificultad: Alta)
(Vinculado a CE a, CE b, CE c, CE e, CE f, CE g)
Parte A: Programación (3 puntos)
Diseña un programa en Java que simule el proceso de recarga de batería de un robot explorador.
El robot empieza con una batería inicial del 10%.
En cada ciclo (bucle), el robot realiza una carga interactiva donde se le pregunta al usuario cuántos puntos de energía desea inyectar (un entero entre 1 y 30).
Si el usuario introduce un valor fuera de rango (menor que 1 o mayor que 30), el sistema debe mostrar "Entrada inválida" y pedir el valor de nuevo sin sumar nada a la batería.
La recarga finaliza cuando la batería alcanza o supera el 100%.
Al finalizar el bucle, debe mostrar el número total de ciclos de recarga que han sido necesarios.

 */
public class Examen2_3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        //Declaramos Variables
        int cargaAct = 10;
        int cargaNue = 0;
        int contador = 1;
        int cargaMax = 100;

        //hacemos el bucle
        while (cargaAct < 100) {
        //Preguntamos cuanta carga queremos meterle
```

```
        System.out.println("Ingrese cuanta carga le añadiras");
        cargaNue = sc.nextInt();
        //Si es menor que 1 o mayor que 30 le dice que no es valida y no hace nada, empieza de nuevo el bucle
        if (cargaNue < 1 || cargaNue > 30) {System.out.println("Entrada no valida");}
        //Si es un valor correcto suma 1 al contador y suma la nueva carga al robot y enseña el % de carga
        else {
        ++contador;
        cargaAct = cargaAct + cargaNue;
        System.out.println("Bateria al " + cargaAct + "%");
        }

        }
        //al estar al 100% o mas se acaba
        System.out.println("Se ha cargado " + contador + " veces");
}
```

} Traza de Código (1 punto)

Analiza el siguiente fragmento de código diseñado para contar cuántos números pares hay entre 1 y 5:

```java
int contadorPares = 0;
int i = 1;

while (i <= 5) {
    if (i % 2 == 0) {
        contadorPares++;
    }
    // Punto de Inspección (Breakpoint)
    i++;
}
System.out.println("Pares encontrados: " + contadorPares);
```

Responde en un documento (puedes copiar la tabla):

1. Completa la **Tabla de Traza** indicando los valores que toman las variables i y contadorPares en el *Punto de Inspección* al final de cada iteración del bucle:

Iteración	Valor de i (después de i++)	Valor de contadorPares

1ª iteración	2	1
2ª iteración	3	2
3ª iteración	4	3
4ª iteración	5	4
5ª iteración	6	5

2. Si en NetBeans colocamos un **Breakpoint** en la línea `if (i % 2 == 0)`, ¿cuántas veces se detendrá la ejecución del programa en ese punto antes de terminar?

Dos veces, en el 1 y en el 2 se para porque 2 es divisible entre 2 y su resto es 0